Deploying Fast and Large Scale Web Applications

Thierry Sans

Users care about speed

"Amazon found every 100ms of latency cost them 1% in sales"

"Google found an extra .5 seconds in search page generation time dropped traffic by 20%"

✓ A fast interface feels responsive and trustworthy

✓ Latency affects conversion, engagement, and accessibility

Three different engineering goals

Performance

How quickly does one request complete?

Scalability

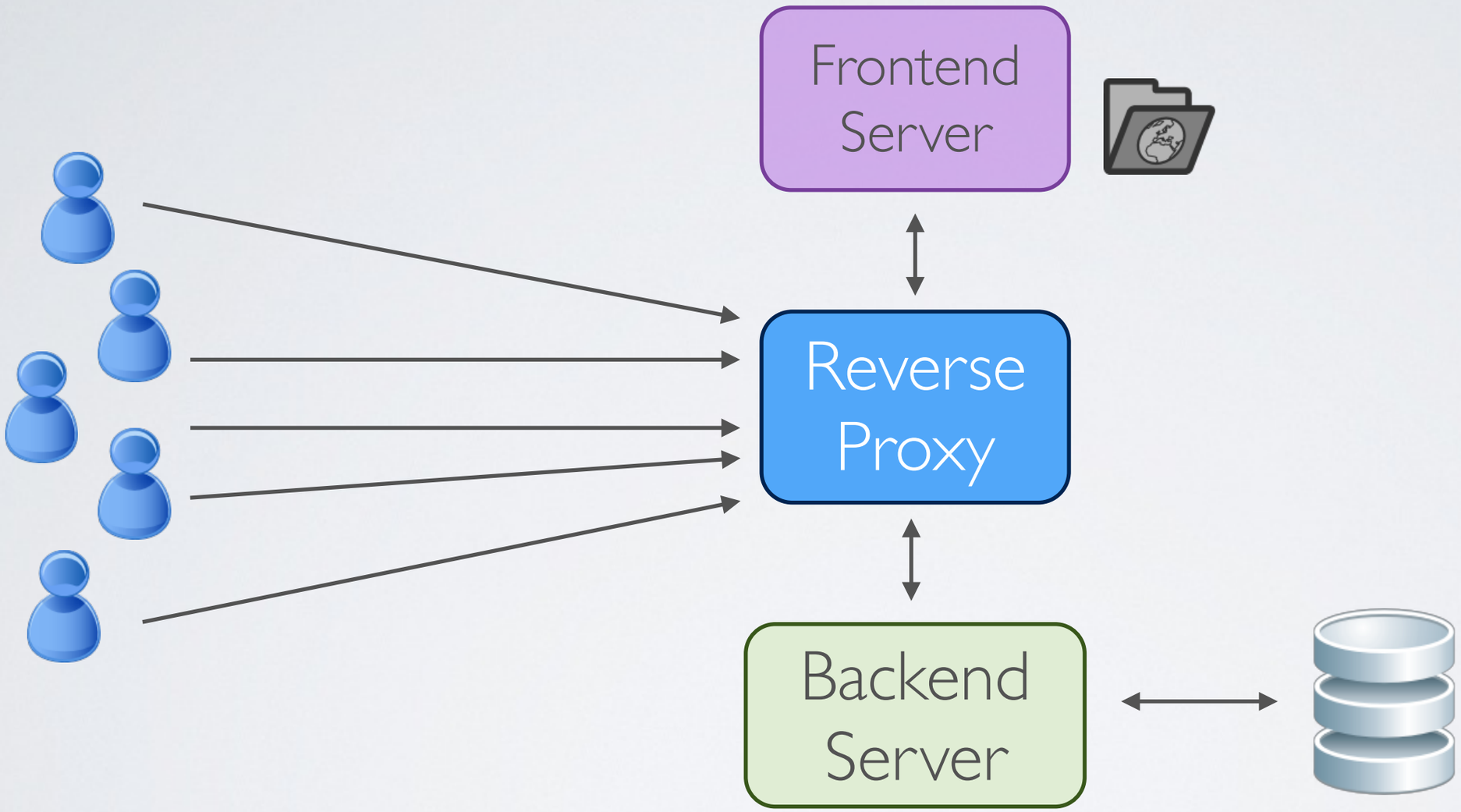
How well does performance hold as load increases?

Reliability

How does the service behave when something fails?

Measuring Performances

Our deployment (so far)



Performance Vocabulary

Latency
Average time per requests

Throughput
Completed requests per second

Tail Outlier
95 Percentile and 99 Percentile expose outliers because throughput average hides the worst experiences

Concurrency
Requests sent concurrently (a.k.a "in flight") at one time per client

Where to optimize?

Problem

Speeding up a non-bottleneck may produce negligible benefit

Example

A backend can reply in 20 ms but requires 700 ms in average for a database request

Solution

Break latency into “spans” instead of guessing

Measure before you optimize

From the Browser

Measure end-to-end performance with network overhead (using *Browser Dev Tools*, *Lighthouse*)

From the Server

Measure time to execute a request (using *K6*, *Locust*)

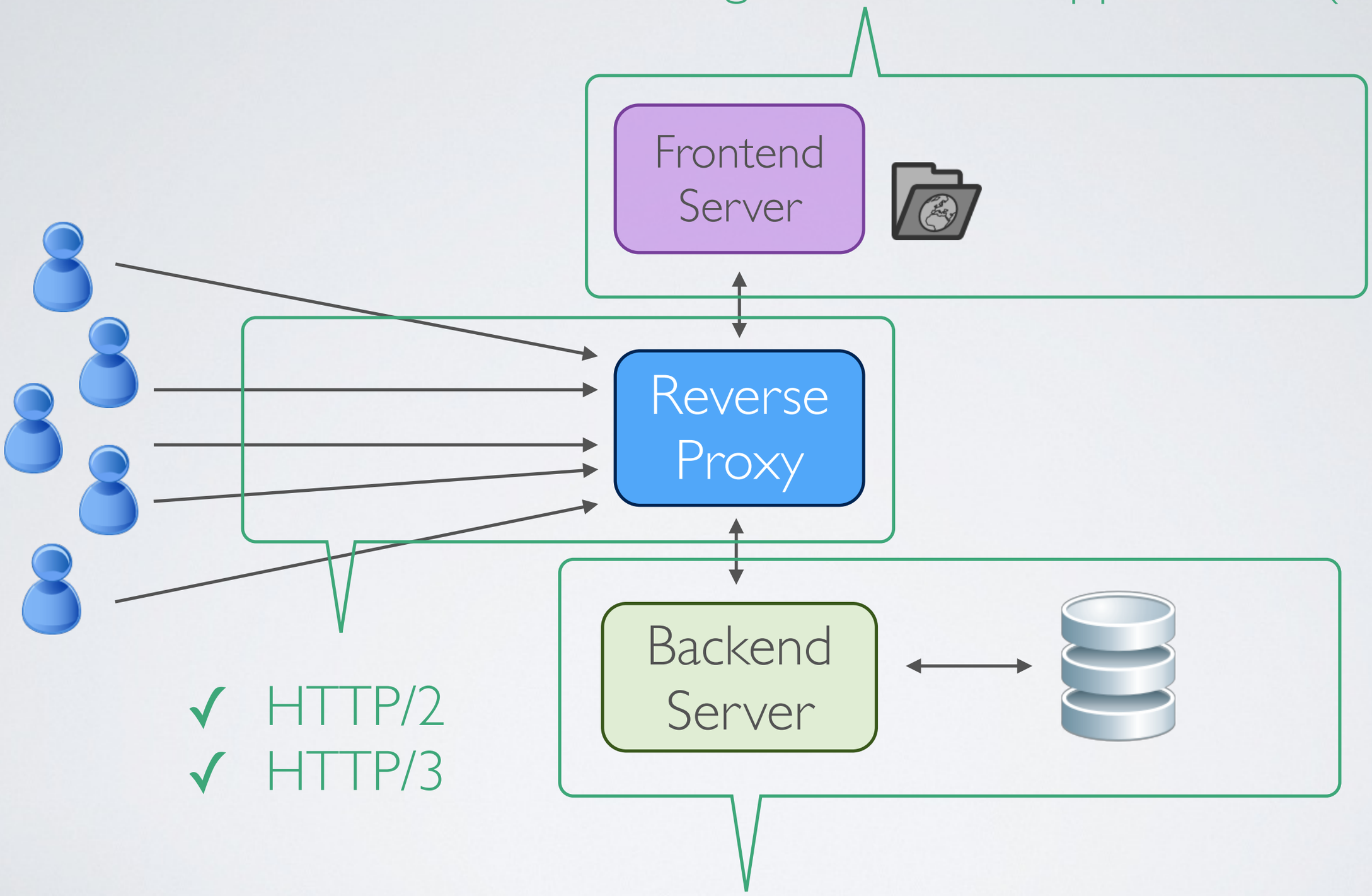
From the Database

Measure time to execute a database query (using database profiling tools)

Improving Performance

Solutions

- ✓ Frontend Build Optimization
- ✓ Browser Cache
- ✓ Progressive Web Applications (PWA)



- ✓ HTTP/2
- ✓ HTTP/3

- ✓ Backend Caching

Improving Performance

Fast Frontend Delivery with HTTP/2 and HTTP/3

HTTP/1 vs HTTP/2 vs HTTP/3

HTTP/1

- Text-based messages
- No protocol-level multiplexing
- Multiple connections often used

HTTP/2

- Binary framing over
- HPACK header compression
- TCP Multiplexed request streams

HTTP/3

- HTTP semantics over QUIC (use of UDP and TLS 1.3)
- Independent QUIC streams
- QPACK header compression

✓ HTTP/2 and HTTP/3 are compatible with the HTTP/1 semantics, **only the delivery changes**

HTTP 1

Every HTTP request/response is sent over its own TCP request

But with **`keep-alive`** header, several request/response can be sent sequentially through the same TCP stream

Problem: no parallelism unless opening several TCP streams

HTTP2

Improvement over HTTP/1:

- **Multiplexing:** multiple requests and responses can travel back and forth concurrently over a single TCP connection
- **Header compression (HPACK):** a predefined list of 61 common HTTP header fields and values referenced by 1-byte decimal indexes (1 through 61)
- ~~**Server-Side Push**~~ **(now retired):** send multiple HTTP responses for a given request that was mainly used to send back JS, CSS and media files after just a single GET request for /

Removing HTTP/2 Server Push from Chrome

Published on Thursday, August 18, 2022 • Updated on Friday, October 14, 2022



Barry Pollard
Web Performance Developer Advocate for Google
Website Twitter GitHub Mastodon

Table of contents

Following on from the previous announcement, support of HTTP/2 Server Push will be disabled by default in Chrome 106 and other Chromium-based browsers in their next releases.

- HTTP/2 Server Push was not widely adopted after few years and was redundant with efficient browser caching

HTTP/3

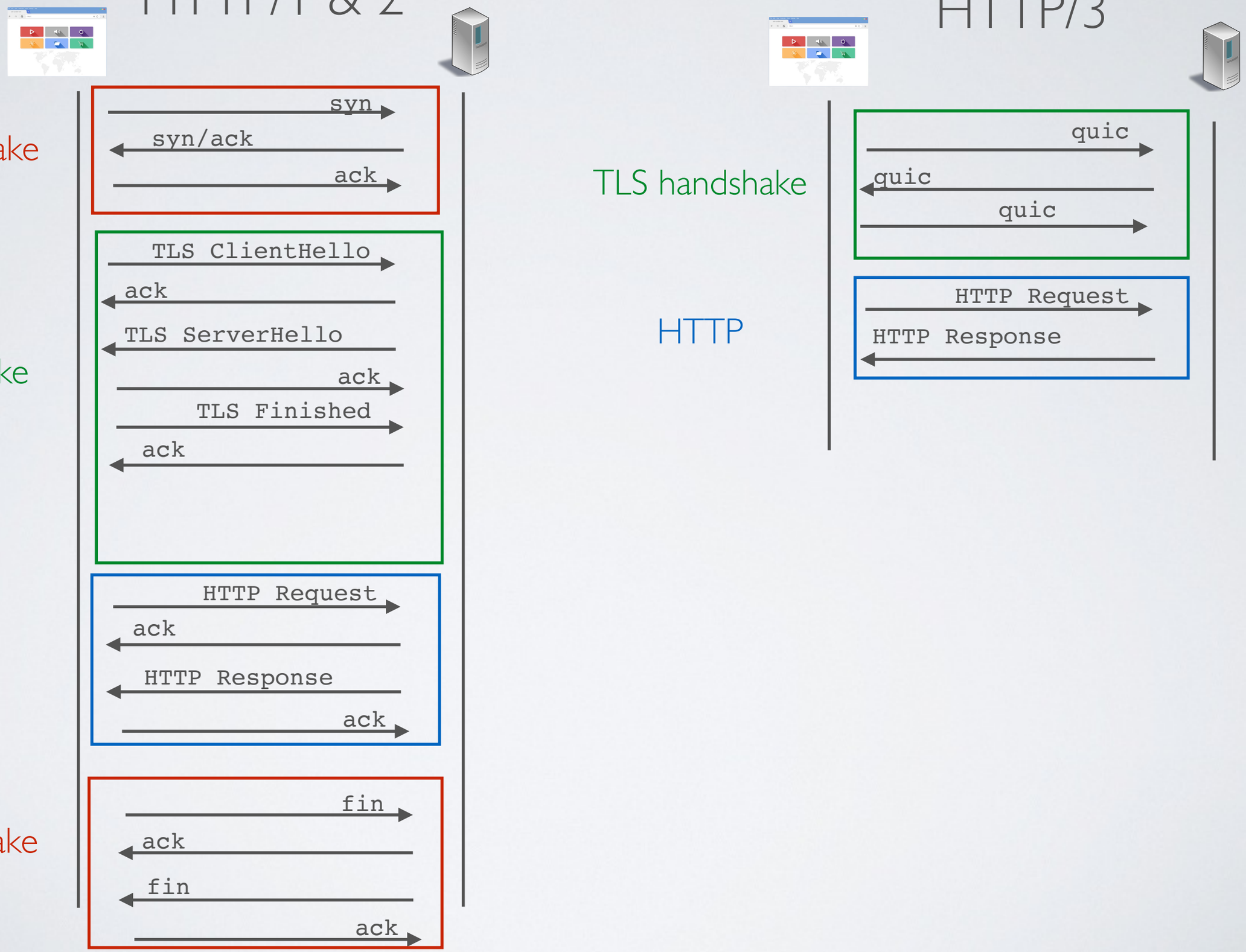
➡ The main change: use **QUIC** (UDP + TLS 1.3) instead of TCP

✓ Faster connection establishment assuming the network connection is stable and reliable

✓ Truly independent request/response streams (faster than HTTP2 multiplexing)

✓ Better header compression (QPACK)

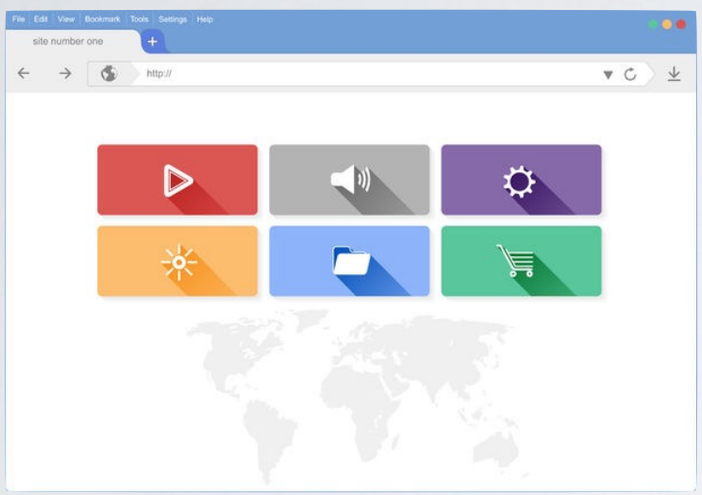
HTTP/3 (standard draft)

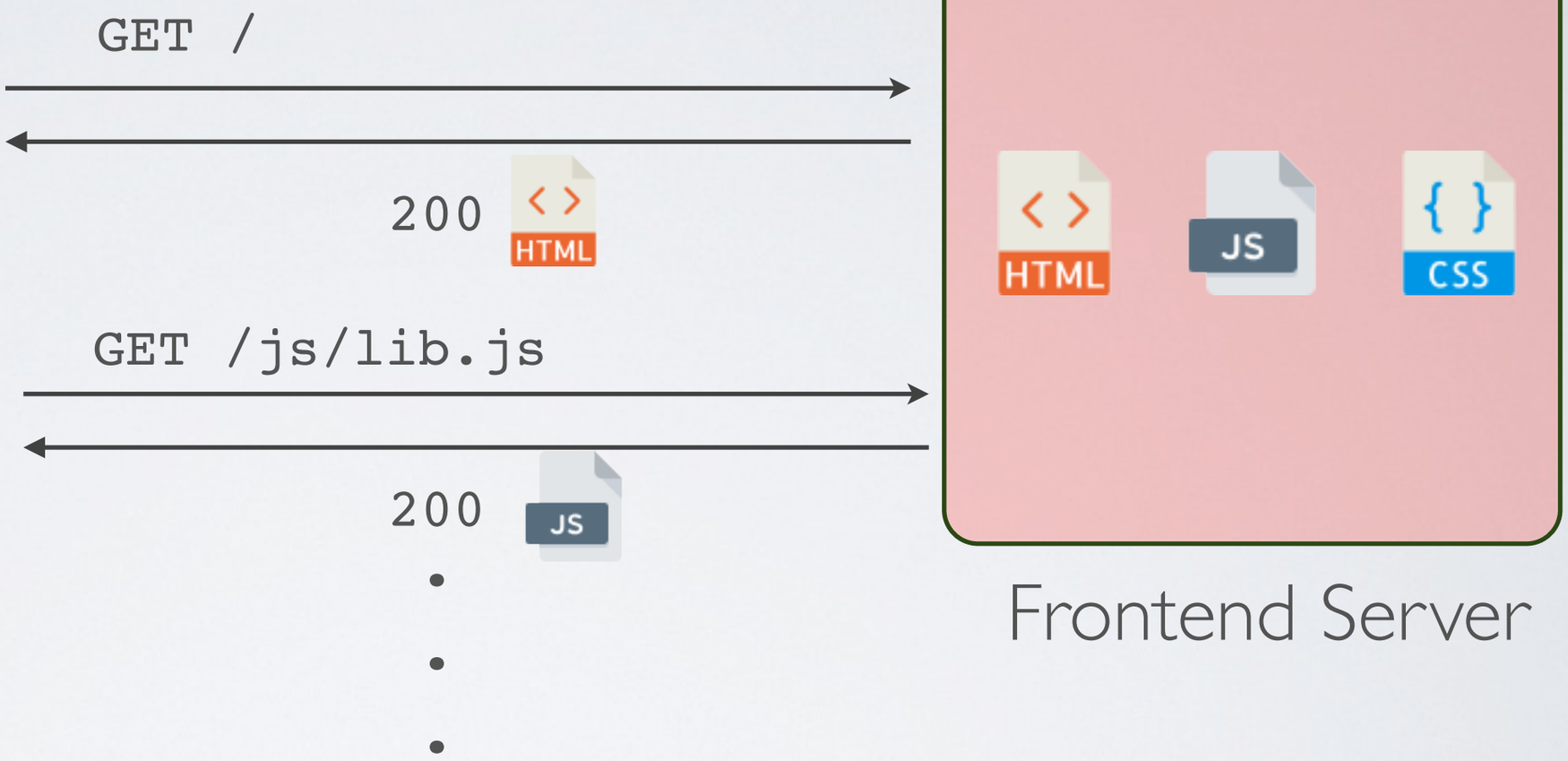


Improving Performance

Browser Caching

The problem

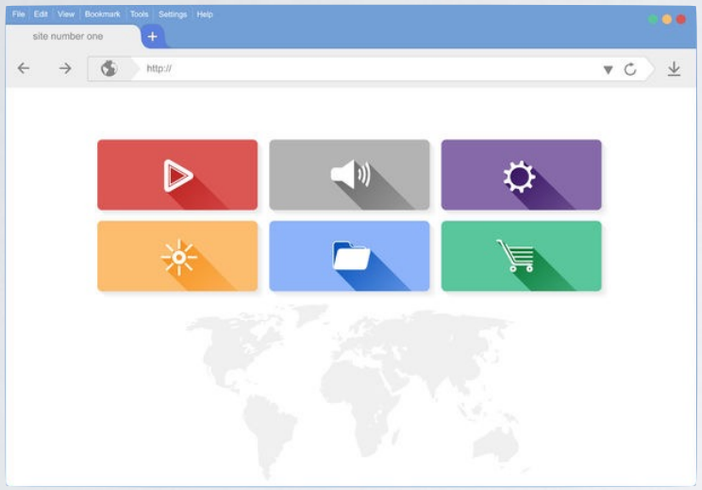




- When visiting the same website, the browser **downloads the same frontend payload** over and over **even if the content has not changed**

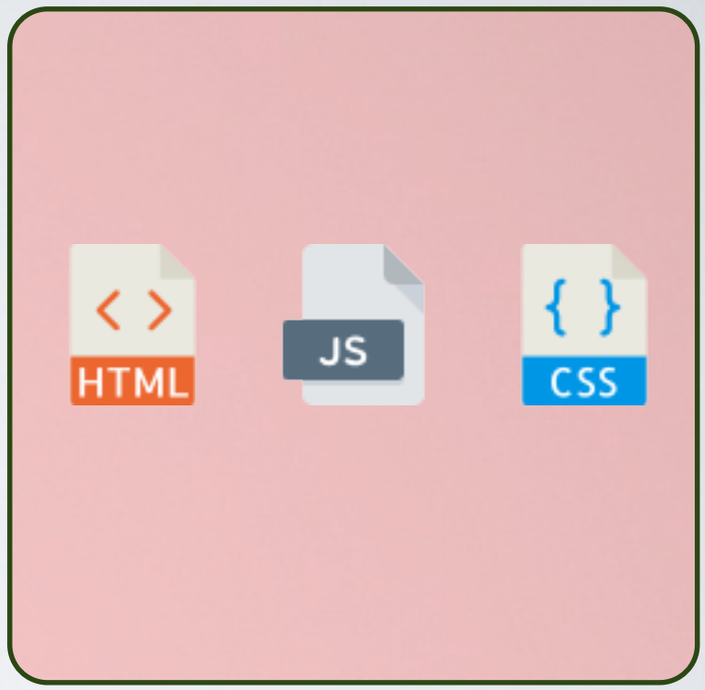
The solution

Browser



GET / eTag: J5LO35OC

304 (Not Modified)



Frontend Server

✓ Cache the content on the browser and download if and only if that content has changed (based on a hash called *ETag*)

Cache-Control

The backend tells the browser when to cache using the response header **`Cache-Control`**

- **`Cache-Control: no-store`**
  The content is not cached by the browser
  e.g. sensitive personalized data

- **`Cache-Control: no-cache`**
  The content is cached and the browser checks whether the server has a new content for a given hash called *ETag* (using **`If-None-Match`** header in request)
  e.g. HTML documents such as **`index.html`**

- **`Cache-Control: public, max-age=31536000, immutable`**
  The content is cached and the browser reused it without checking for an update with the server until **`max-age`** has expired
  e.g. Hashed JS/CSS/Media assets (see optimizing frontend code)

Improving Performance

Optimizing Frontend Code

Frontend Build Optimization

- **Minification and tree shaking**
  to remove unnecessary code and reduce the files sizes

- **Code splitting and dynamic imports**
  to load features only when needed (native with Javascript ES6 modules)

- **Brotli/Gzip compression**
  to reduce transferred bytes

- **Content-hashed assets**
  to enable long-lived browser caching
  (using **`Cache-Control: public`** seen earlier)

- **Optimization of images, fonts, and critical CSS**
  to minimize transferred bytes

✓ Use open-source **`esbuild`** tool JavaScript and CSS bundler and minifier

Improving Performance

Content Delivery Network

Content Delivery Network (CDN)

A CDN is a geographically distributed network of servers that delivers web content from locations close to users

- **Cache hit:** content is served directly from an edge server, reducing latency and origin load

- **Cache miss:** the edge server retrieves the content from the origin, potentially caching it for future requests

➡ Typically static assets (HTML, CSS, JavaScript, images, videos), but also cacheable API responses

Example: *Akamai*, *Cloudflare*

Pors and Cons of CDN

Benefits

- ✓ Lower latency for geographically distributed users
- ✓ Reduced bandwidth and computational load on origin servers
- ✓ Improved scalability and availability; many CDNs also provide DDoS protection

Limitations

- ◉ Cache invalidation and stale content; personalized responses must not be inadvertently shared between users
- ◉ Cost

PWA

Progressive Web Applications

The idea

➡ A PWA uses web technologies to provide an application-like experience

- A web app manifest provides installation metadata
- A Service Worker can intercept network requests and implement offline caching using the Cache Storage API

Improving Performance

Backend Caching

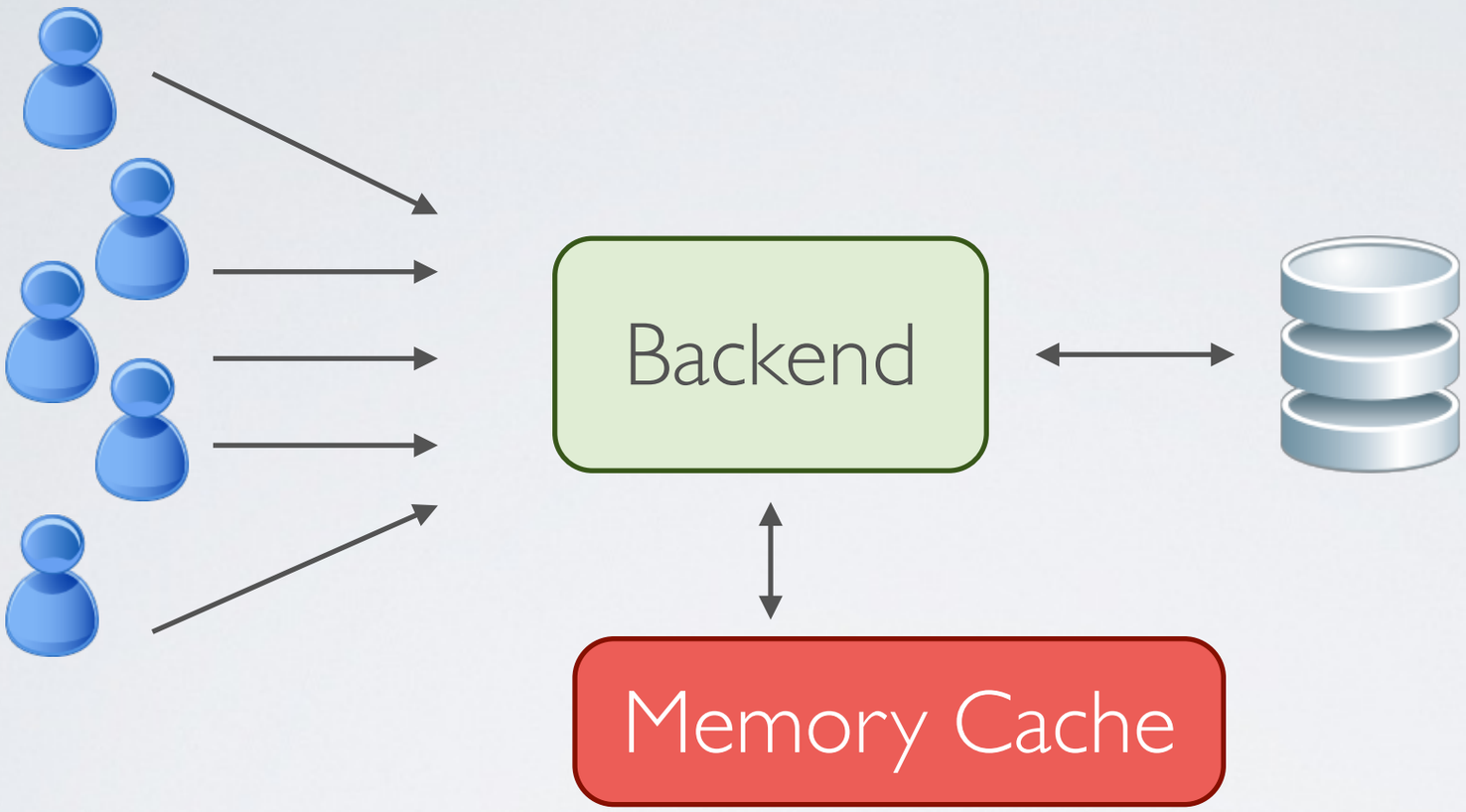
How to improve response time on the backend?

Processing the request means:

1. Parse the HTTP request
2. Map the URL to the handler
3. Query the database or third-party API
4. Compute the HTTP response

DB and API accesses are expensive (in time but also money when your host charges you each access)

Fine-grained caching with the web application



Cache controlled by the program

- ◉ Specific for each app
- ✓ Good for caching database requests and storing sessions
- ➡ Popular memory cache : *Memcached / Redis*

Distributed Shared Cache : Memcached

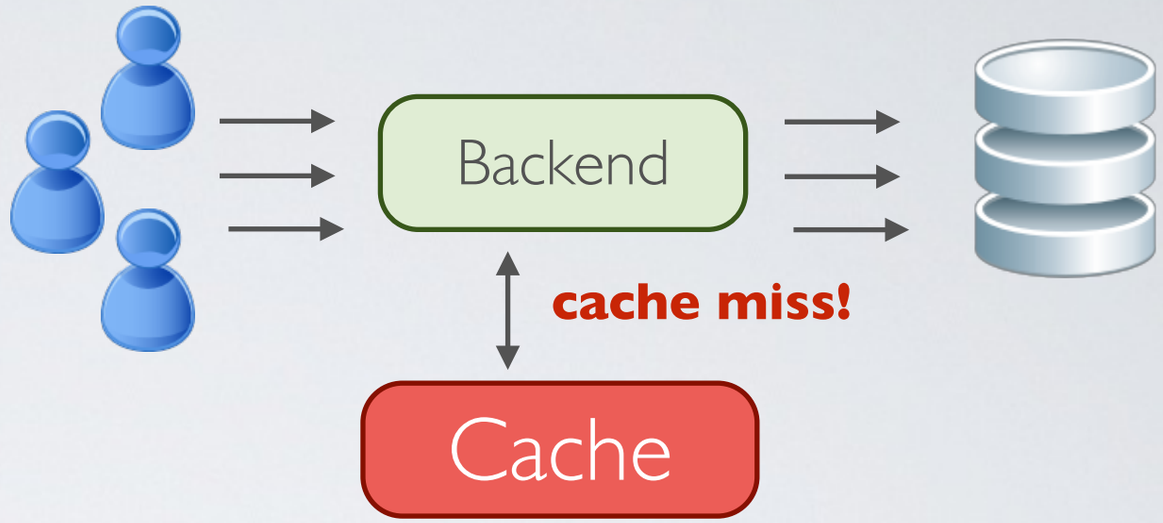
http://memcached.org/

- Store key/value pairs in memory
- Throw away data that is the least recently used

A typical cache algorithm

```
retrieve from cache
if data not in cache:
   # cache miss
   query the database or API
   update the cache
return result
```

Cache Stampede (a.k.a dog piling)



Problem:

Multiple concurrent requests doing the same request because cache was cleared

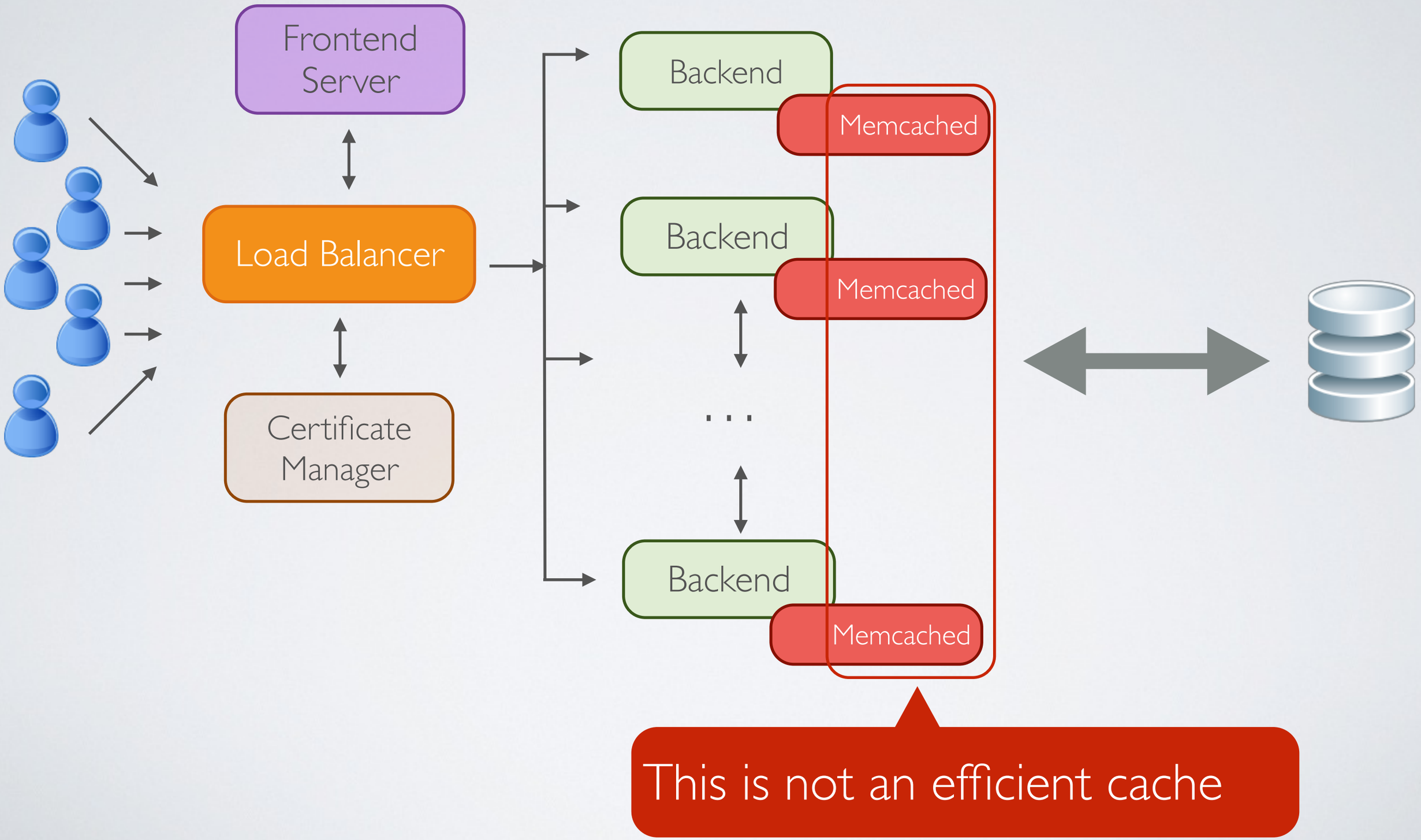
Solution:

- update the cache instead of clearing it after an insert
- a page view will never query the database

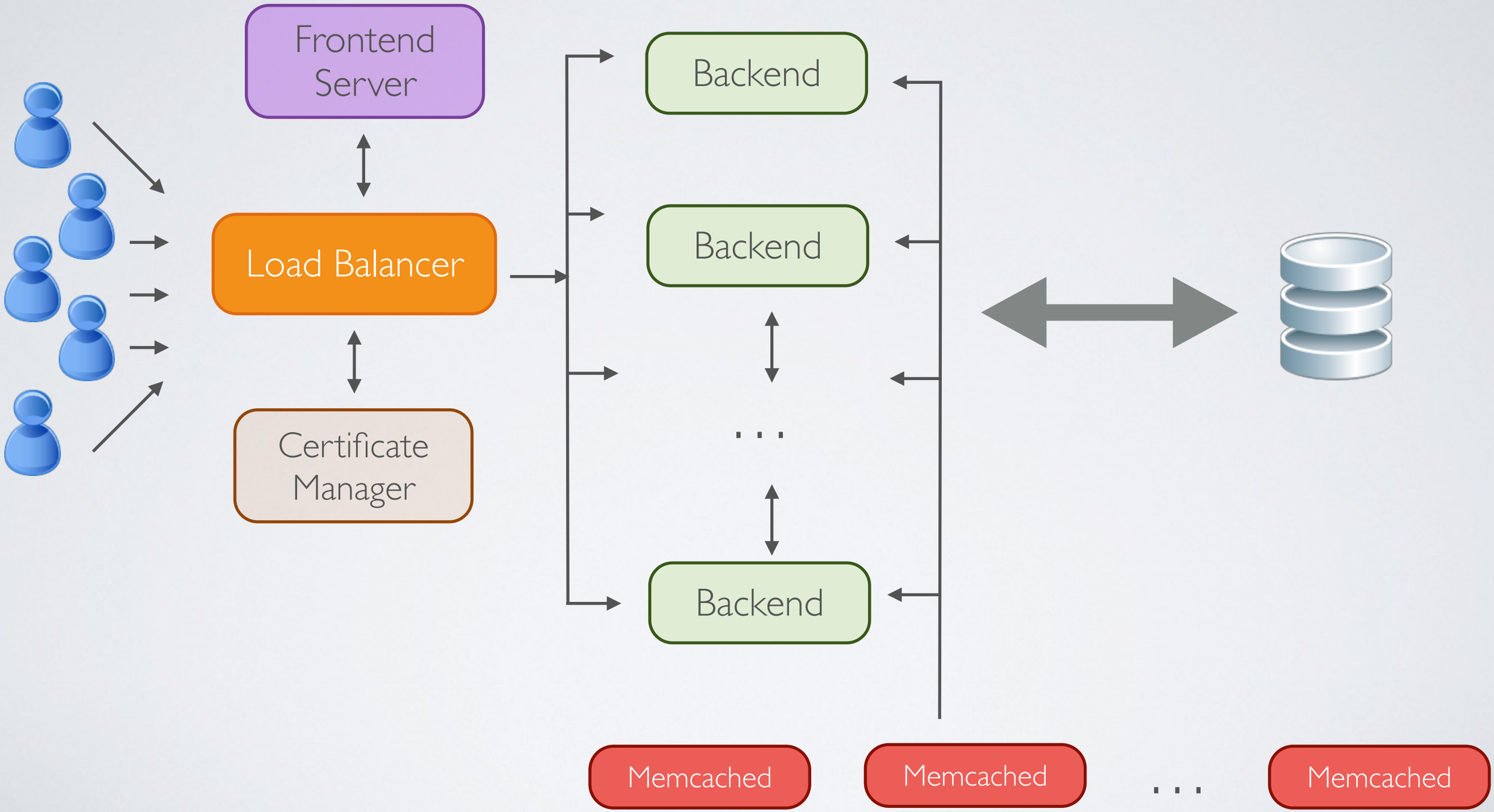
➡ Requires cache warming

Scaling

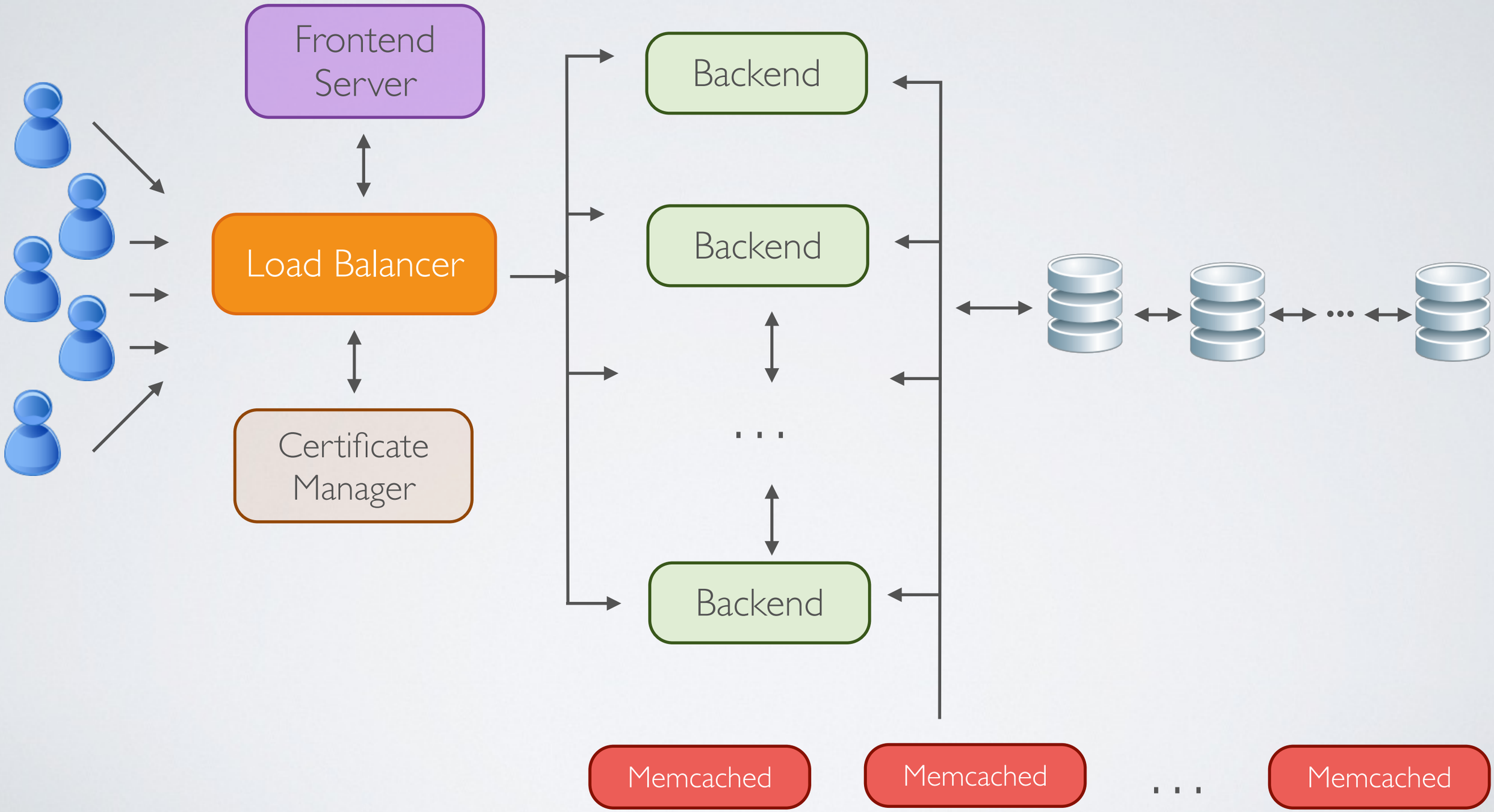
Serving multiple apps with a load balancer



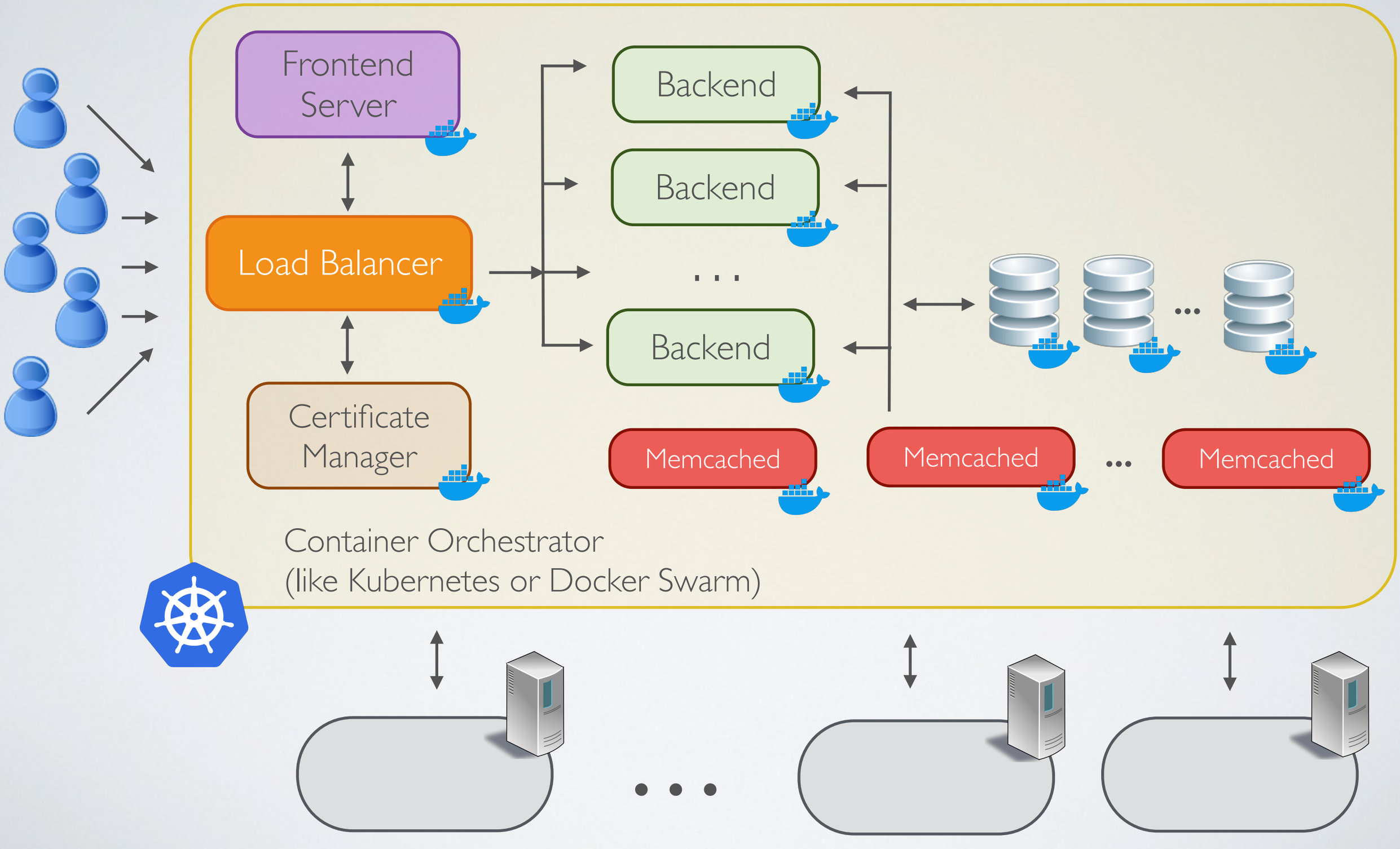
Distributed Shared Cache



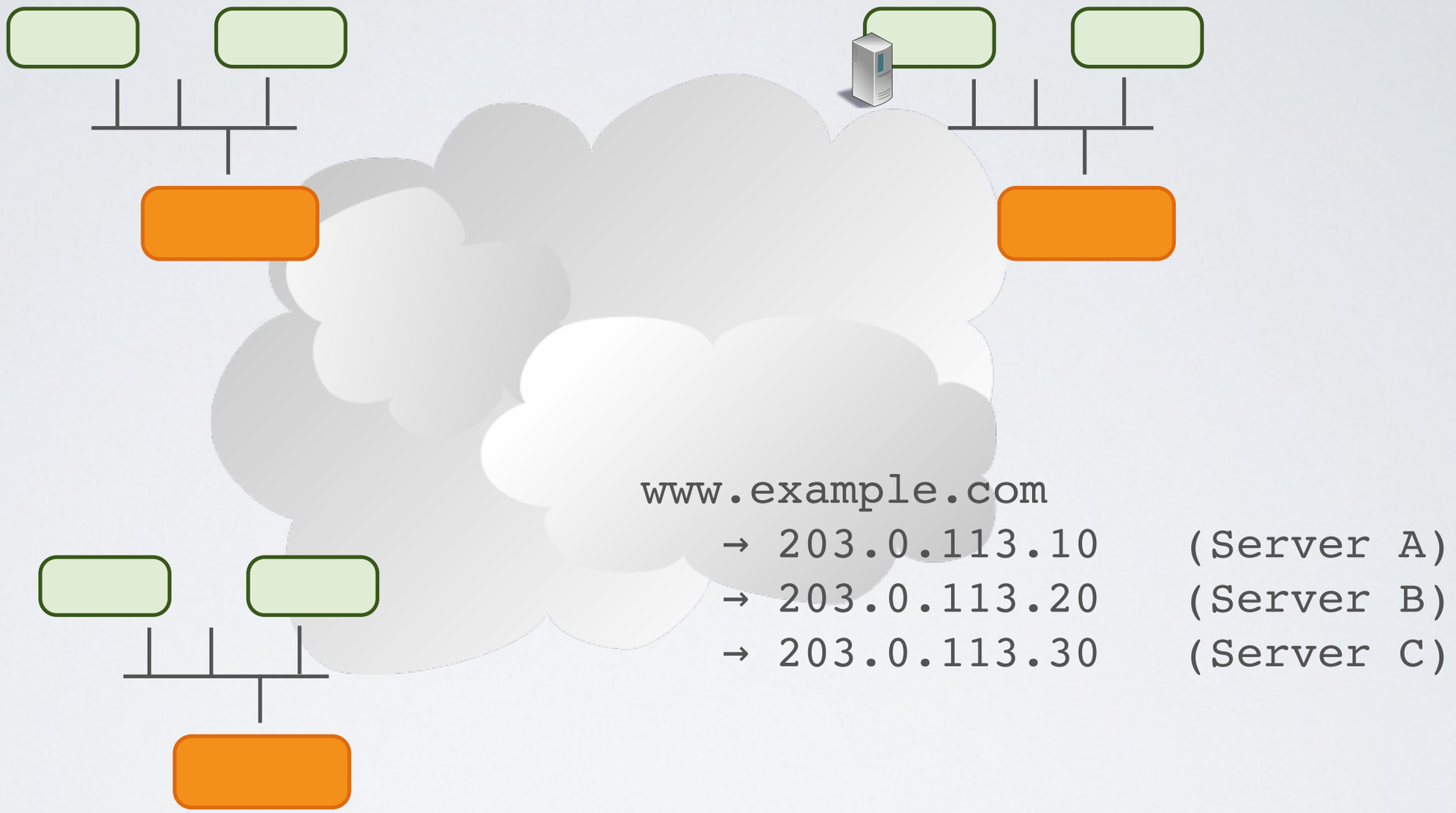
Database Sharding



Automatic Scaling with container Orchestration



DNS Load Balancing



✓ DNS load balancing distributes clients across multiple servers by returning different IP addresses for the same domain name

Pors and Cons of DNS Load Balancing

Benefits

- ✓ Simple, scalable, and can distribute traffic across geographic regions
- ✓ No centralized load balancer in the application data path

Limitations

- ◉ DNS cannot guarantee an even distribution of requests
- ◉ DNS operates at name-resolution time, not on individual HTTP requests